

Operating System Principles

操作系统原理

Process/Thread Scheduling

李旭东

leexudong@nankai.edu.cn

Nankai University

Objectives

- Scheduler
- Process Behavior
- Scheduling Mode
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling

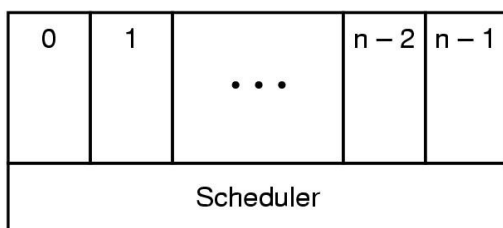
leexudong@nankai.edu.cn

2

MultiProgramming

- Scheduler
- Scheduling algorithm

Processes



leexudong@nankai.edu.cn

3

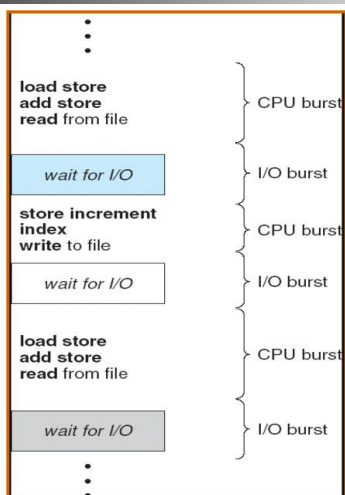
Scheduler

- Short-term Scheduler
 - CPU
- Middle-term Scheduler
 - Memory
- Long-term Scheduler
 - Job

leexudong@nankai.edu.cn

4

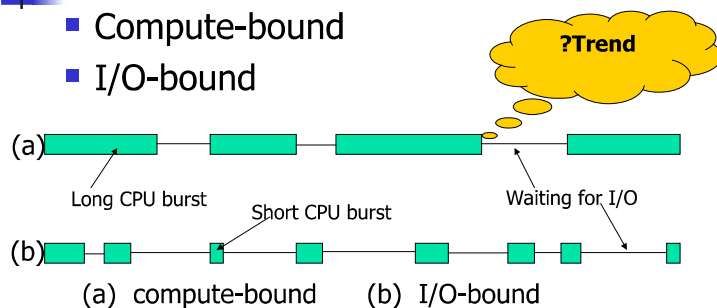
CPU And I/O Bursts in a Process



5

Process Behavior

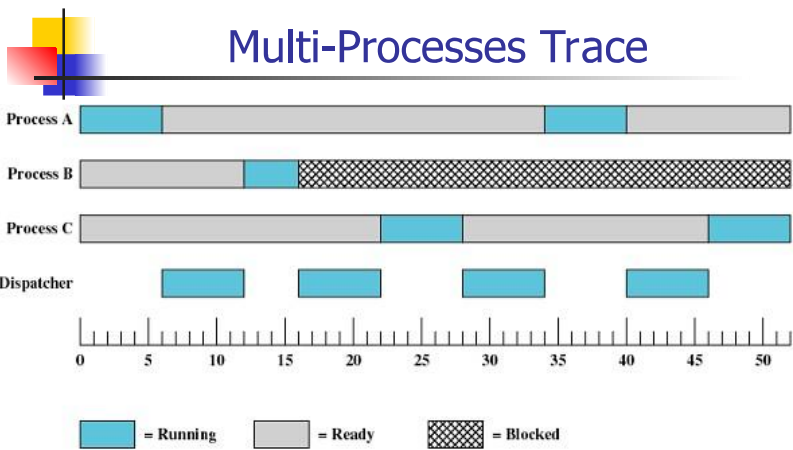
- Compute-bound
- I/O-bound



leexudong@nankai.edu.cn

6

Multi-Processes Trace



leexudong@nankai.edu.cn

7

When to Schedule

- A new process is created
- A process exits
- A process blocks on I/O, on a semaphore, or for some other reason
- An I/O interrupt occurs

leexudong@nankai.edu.cn

8

Dispatcher

- A module that gives control of the CPU to the process selected by the short-term scheduler
 - Switching context
 - Switching to user mode
 - Jumping to the proper location in the user program to restart that program
- Dispatch latency 调度延迟
 - The time it takes for the dispatcher to stop one process and start another running

leexudong@nankai.edu.cn

9

Scheduling Modes

- Preemptive
 - 抢占式
- Nonpreemptive
 - 非抢占式, 非剥夺式

leexudong@nankai.edu.cn

10

Categories of Scheduling Algorithms

- Batch
- Interactive
- Real-time

leexudong@nankai.edu.cn

11

Scheduling Criteria

- CPU utilization 利用率
- Throughput 吞吐量
- Turnaround time 周转时间
 - Waiting to get into memory
 - Waiting in the ready queue
 - Executing on the CPU
 - Doing I/O
- Waiting time
- Response time
- ...

leexudong@nankai.edu.cn

12

Scheduling Algorithm Goals

All systems

Fairness - giving each process a fair share of the CPU
 Policy enforcement - seeing that stated policy is carried out
 Balance - keeping all parts of the system busy

Batch systems

Throughput - maximize jobs per hour
 Turnaround time - minimize time between submission and termination
 CPU utilization - keep the CPU busy all the time

Interactive systems

Response time - respond to requests quickly
 Proportionality - meet users' expectations

Real-time systems

Meeting deadlines - avoid losing data
 Predictability - avoid quality degradation in multimedia systems

leexudong@nankai.edu.cn

14

First-come first-served

Average waiting time

Process	Burst Time
P_1	24
P_2	3
P_3	3

P_1 24
 P_2 3
 P_3 3



$$Awt = (0 + 24 + 27) / 3 = 17$$



Awt=?

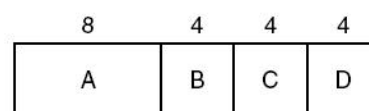
leexudong@nankai.edu.cn

15

Scheduling in Batch System

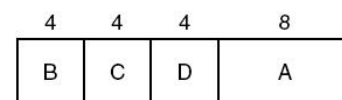
- First-come first-served
- Shortest job first
- Shortest remaining Time next

Shortest job first



(a)

- nonpreemptive
- preemptive



(b)

Figure 2-40. An example of shortest job first scheduling.
 (a) Running four jobs in the original order.
 (b) Running them in shortest job first order.

leexudong@nankai.edu.cn

16

Quiz

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

P_1 6
 P_2 8
 P_3 7
 P_4 3

SJF: AWT=?

FCFS: AWT=?

leexudong@nankai.edu.cn

17

Shortest job first

- How to predict length of the next CPU burst?

- exponential average

$$T_{n+1} = at_n + (1-a)T_{n-1}, \quad 0 \leq a \leq 1$$

t_n the length of the n th CPU burst

T_{n+1} the predicted value of the next CPU burst

$$T_{n+1} = at_n + (1-a)at_{n-1} + \dots + (1-a)^j at_{n-j} + \dots + (1-a)^{n+1} T_0$$

leexudong@nankai.edu.cn

18

Shortest remaining Time next

- i.e. Preemptive SJF scheduling

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Nonpreemptive SJF scheduling: AWT=?

Preemptive SJF scheduling: AWT=?

Scheduling in Interactive System

- Round-Robin Scheduling
- Priority Scheduling
- Multiple Queues
- Shortest Process Next
- Guaranteed Scheduling
- Lottery Scheduling
- Fair-Share Scheduling

Round-Robin Scheduling

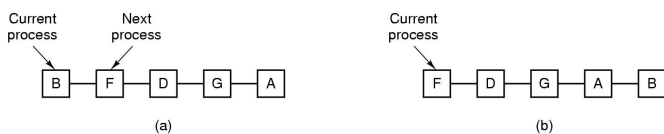


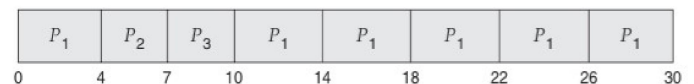
Figure 2-41. Round-robin scheduling.
(a) The list of runnable processes.

(b) The list of runnable processes after B uses up its quantum.

Round-Robin Scheduling

Process	Burst Time
P_1	24
P_2	3
P_3	3

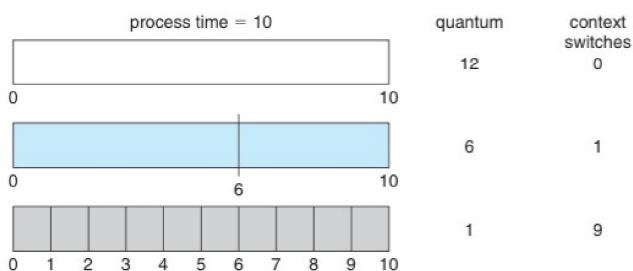
a time quantum of 4 milliseconds



$$AWT = (6 + 4 + 7) / 3 = 5.66$$

Quantum Value

- How a smaller time quantum increases context switches



Priority Scheduling

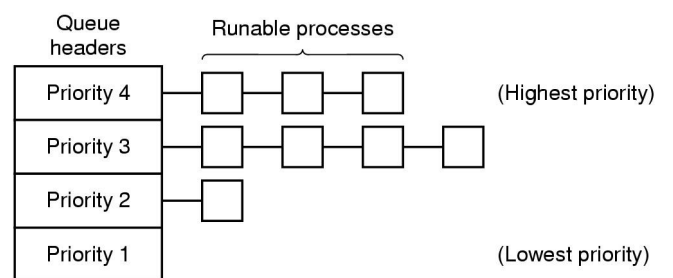
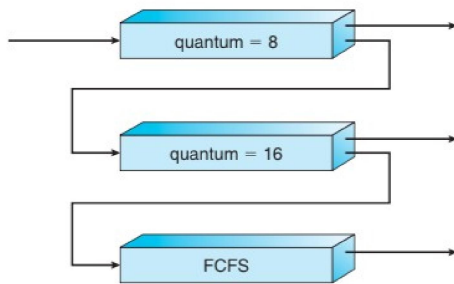


Figure. A scheduling algorithm with four priority classes.

Multilevel Feedback Queue Scheduling

- Idea
 - Separate processes with different CPU-burst characteristics
 - Allow a process to move between queues

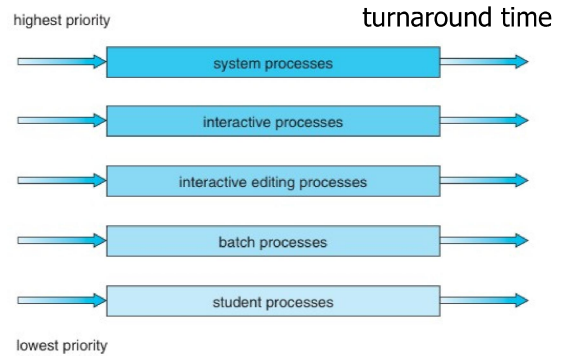


leexudong@nankai.edu.cn

25

Multilevel Queue Scheduling

- Foreground (interactive) processes
- Background (batch) processes



leexudong@nankai.edu.cn

26

Multilevel Feedback Queue Scheduling

- the scheduler is defined by the following parameters:
 - The number of queues
 - The scheduling algorithm for each queue
 - The method used to determine when to upgrade a process to a higher-priority queue
 - The method used to determine when to demote a process to a lower-priority queue
 - The method used to determine which queue a process will enter when that process needs service

leexudong@nankai.edu.cn

27

Lottery Scheduling



leexudong@nankai.edu.cn

28

More Scheduling Algorithms

- Guaranteed Scheduling
- Fair-Share Scheduling
- ...

leexudong@nankai.edu.cn

29

Scheduling in Real-time System

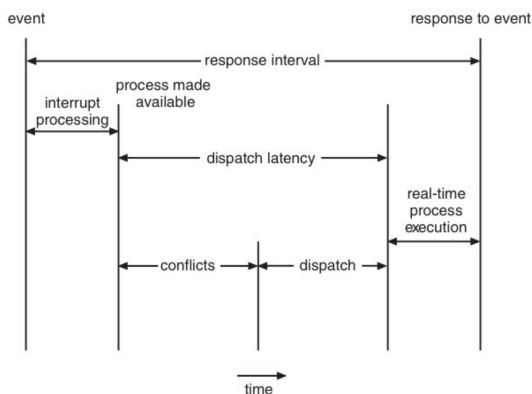
- Categories I
 - Hard real time
 - Soft real time
- Categories II
 - Periodic
 - Aperiodic
- Categories III
 - Static
 - dynamic

leexudong@nankai.edu.cn

30

Real-time CPU Scheduling

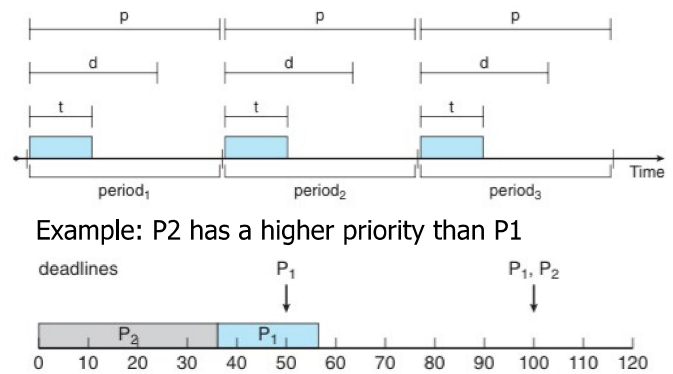
- Minimizing Latency 最小化延迟
 - Interrupt latency, Dispatch latency



31

Real-time CPU Scheduling

- Priority-Based Scheduling



leexudong@nankai.edu.cn

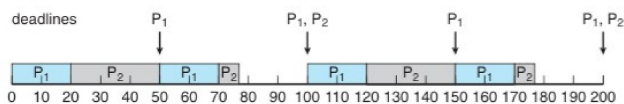
32

Real-time CPU Scheduling

- Rate-Monotonic Scheduling
 - 单一速率
 - a static priority policy with preemption

Example:

P1 a higher priority than P2;
the period of P1 is shorter than that of P2

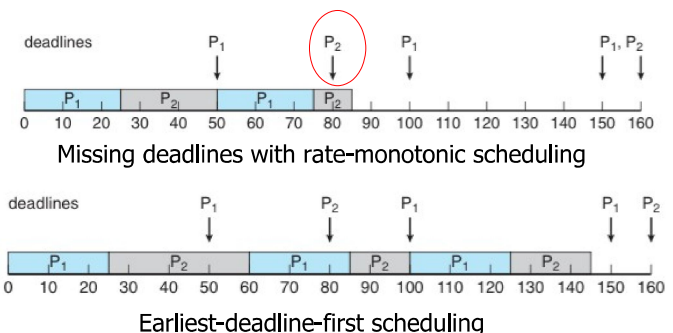


leexudong@nankai.edu.cn

33

Real-time CPU Scheduling

- Earliest-Deadline-First Scheduling
 - dynamically assigns priorities according to deadline



leexudong@nankai.edu.cn

34

Real-time CPU Scheduling

- Proportional 成比例的 Share Scheduling
 - Proportional share schedulers operate by allocating T shares among all applications
 - An application can receive N shares of time, thus ensuring that the application will have N/T of the total processor time

leexudong@nankai.edu.cn

35

Real-time CPU Scheduling

- POSIX Real-Time Scheduling
 - SCHED_FIFO
 - SCHED_RR
 - SCHED_OTHER
 - pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)
 - pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)

leexudong@nankai.edu.cn

36

Multiple-Processor Scheduling

- multiple CPUs
 - load sharing becomes possible—but scheduling problems become correspondingly more complex
- Approaches to Multiple-Processor Scheduling
 - asymmetric multiprocessing
 - all scheduling decisions, I/O processing, and other system activities handled by a **single processor**—the master server.
 - The other processors execute only user code.
 - symmetric multiprocessing (SMP)
 - Each processor is self-scheduling. All processes may be in a common ready queue, or each processor may have its own private queue of ready processes.

leexudong@nankai.edu.cn

37

Multiple-Processor Scheduling

- Processor Affinity 处理器亲和性
 - Consider what happens to cache memory when a process has been running on a specific processor. The data most recently accessed by the process populate the cache for the processor.
 - As a result, successive memory accesses by the process are often satisfied in cache memory.
- Deference Forms of Processor Affinity
 - soft affinity
 - When an operating system has a policy of attempting to keep a process running on the same processor—but not guaranteeing that it will do so
 - hard affinity
 - sched_setaffinity() system call

leexudong@nankai.edu.cn

38

Linux Scheduler

- /kernel/sched/core.c
 - static void __sched notrace __schedule(bool preempt)
 - static __always_inline struct rq * context_switch(struct rq *rq, struct task_struct *prev, struct task_struct *next, struct rq_flags *rf)
 - static inline struct task_struct * pick_next_task(struct rq *rq, struct task_struct *prev, struct rq_flags *rf)

leexudong@nankai.edu.cn

39

Linux Scheduler (cont.,)

```

1193 }
1194
1195 /*
1196  * Pick up the highest-prio task:
1197  */
1198 static inline struct task_struct *
1199 pick_next_task(struct rq *rq, struct task_struct *prev, struct rq_flags *rf)
1200 {
1201     const struct sched_class *class;
1202     struct task_struct *p;
1203
1204     /*
1205      * Optimization: we know that if all tasks are in the fair class we can
1206      * call that function directly, but only if the above task wasn't of a
1207      * higher scheduling class, because otherwise there is no
1208      * opportunity to pull in more work from other CPUs.
1209      */
1210     if (likely(prev->sched_class == &idle_sched_class))
1211         prev->sched_class = &fair_sched_class;
1212     rq->nr_running = rq->cpu_nr_running;
1213     p = fair_sched_class.pick_next_task(rq, prev, rf);
1214     if (unlikely(p == RETRY_TASK))
1215         goto again;
1216
1217     /* Assumes fair_sched_class->next == &idle_sched_class */
1218     if (unlikely(p))
1219         p = &idle_sched_class.pick_next_task(rq, prev, rf);
1220     return p;
1221 }
1222
1223 again:
1224
1225 for_each_class(class) {
1226     p = class->pick_next_task(rq, prev, rf);
1227     if (p) {
1228         if (unlikely(p == RETRY_TASK))
1229             goto again;
1230         return p;
1231     }
1232 }
1233
1234 /* The idle class should always have a runnable task: */
1235 BUG();
1236 }
1237
1238 /*
1239  * __schedule() is the main scheduler function.
1240  */
1241 #endif
    
```

powered by Blue42

Thread Scheduling

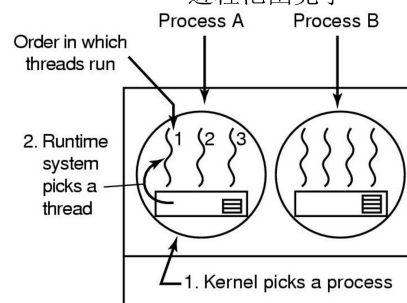
- Two levels of parallelism
- Thread scheduler
 - User-level thread
 - Kernel-level thread
 - (Hyper Thread)

leexudong@nankai.edu.cn

41

Thread Scheduling

User-level thread: **process-contention scope (PCS)**
进程范围竞争



Possible: A1, A2, A3, A1, A2, A3
Not possible: A1, B1, A2, B2, A3, B3

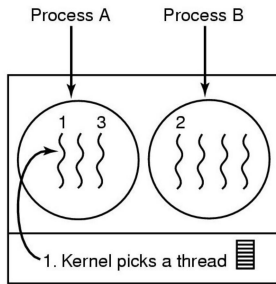
(a) Possible scheduling of user-level threads with a 50-msec process quantum and threads that run 5 msec per CPU burst.

leexudong@nankai.edu.cn

42

Thread Scheduling

Kernel-level thread: **system-contention scope (SCS)**



Possible: A1, A2, A3, A1, A2, A3
Also possible: A1, B1, A2, B2, A3, B3

(b) Possible scheduling of kernel-level threads with the same characteristics as (a).

leexudong@nankai.edu.cn

43

Thread Scheduling Case

```
#include <pthread.h> #include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[]) {
    int i;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;
    /* get the default attributes */
    pthread_attr_t init(&attr);
    /* set the scheduling algorithm to PROCESS or SYSTEM */
    pthread_attr_t setscope(&attr, PTHREAD_SCOPE_SYSTEM);
    /* set the scheduling policy - FIFO, RT, or OTHER */
    pthread_attr_t setschedpolicy(&attr, SCHED_OTHER);
    /* create the threads */
    for (i = 0; i < NUM_THREADS; i++)
        pthread_create(&tid[i], &attr, runner, NULL);
```

leexudong@nankai.edu.cn

44

Thread Scheduling Case

```
/* now join on each thread */
for (i = 0; i < NUM_THREADS; i++)
    pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
    printf("I am a thread\n");
    pthread_exit(0);
}
```

leexudong@nankai.edu.cn

45

Thread Scheduling

- Pthread Scheduling
 - PTHREAD_SCOPE_PROCESS schedules threads using PCS scheduling
 - PTHREAD_SCOPE_SYSTEM schedules threads using SCS scheduling
- pthread_attr_t setscope(pthread_attr_t *attr, int scope)
- pthread_attr_t getscope(pthread_attr_t *attr, int *scope)

leexudong@nankai.edu.cn

46

Policy v.s. Mechanism

- Scheduling mechanism 调度机制
- Scheduling policy 调度策略



leexudong@nankai.edu.cn

47

Summary

- Scheduler
- Process Behavior
- Scheduling Mode
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- ...

leexudong@nankai.edu.cn

48

[illegible]